Universidade Federal de Santa Maria
Departamento de Computação Aplicada
Prof. Cesar Tadeu Pozzer
pozzer@inf.ufsm.br
04/10/2023

1. Teoria dos Grafos

Grafo é uma ferramenta muito comum em jogos. Pode ser utilizado para permitir um personagem viajar de um ponto a outro de modo eficiente, para decidir a próxima estratégia em um jogo ou para resolver um puzzle. O uso mais comum de grafos é para representar a rede de caminhos que um personagem pode navegar no ambiente.

Na Figura 1 são apresentados vários exemplos de grafos. Os grafos podem ser conexos (Figuras A, B, C e D) ou não conexos (Figuras E e F). Um grafo é dito conexo quando se pode traçar um caminho que parte de qualquer nó e chega a qualquer outro. Um grafo é dito completo quando há uma aresta entre cada par de seus vértices. Se as arestas do grafo são orientadas, o grafo é chamado orientado. Para uma lista completa de definições, consulte [1].

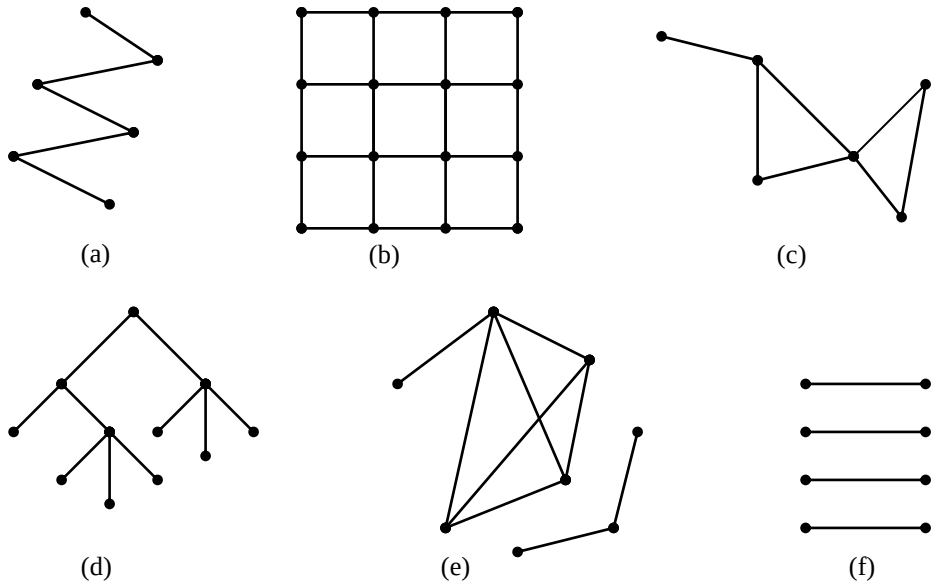


Figura 1: Exemplos de Grafos. Somente os grafos A, B, C e D são conexos

1.1. Notação Formal

Um grafo G pode ser formalmente definido como um conjunto de nós ou vértices, V, ligados por um conjunto de arestas A. Isso é geralmente escrito na forma:

$$G = \{V, A\}$$

Se os nós de um grafo forem numerados com um valor inteiro em um intervalo de 0 a (N-1), uma aresta pode ser referenciada pelos nós que ela conecta, como por exemplo 3-5.

Muitos grafos possuem pesos associados às arestas. Esse peso pode representar o custo necessário para mover de um ponto a outro do grafo. Esse custo pode ser dado em função da distância entre os vértices ou pela

dificuldade de locomoção. Em jogos, regiões montanhosas ou que são mais propensas a ataques de inimigos podem ter um custo maior que regiões planares e com abrigo contra ataques de inimigos. Durante a fase de criação de cada cenário, o próprio game designer pode atribuir pesos diferenciados a cada região do mapa, ou mais especificamente aos caminhos pré-estipulados.

1.2. Árvores

Árvores são um subconjunto dos grafos, visto que em uma árvore, existe um único caminho que leva a qualquer nó, ou seja, não há possibilidade de se voltar a um nó já visitado a partir de seus filhos (não possui ciclos). Na Figura 1, os exemplos A e D representam árvores, apesar de A não ter uma forma que geralmente caracteriza uma árvore. O exemplo F caracteriza uma floresta de árvores.

1.3. Densidade de Grafos

A razão entre os vértices e os nós caracteriza se o grafo é denso ou esparso. Grafos esparsos têm poucas conexões por nó e grafos densos muitas. Na Figura 2 são apresentados exemplos. Para reduzir o uso de CPU, deve-se preferencialmente usar grafos esparsos, especialmente quando se está trabalhando com busca de caminhos (*Path-finding*).

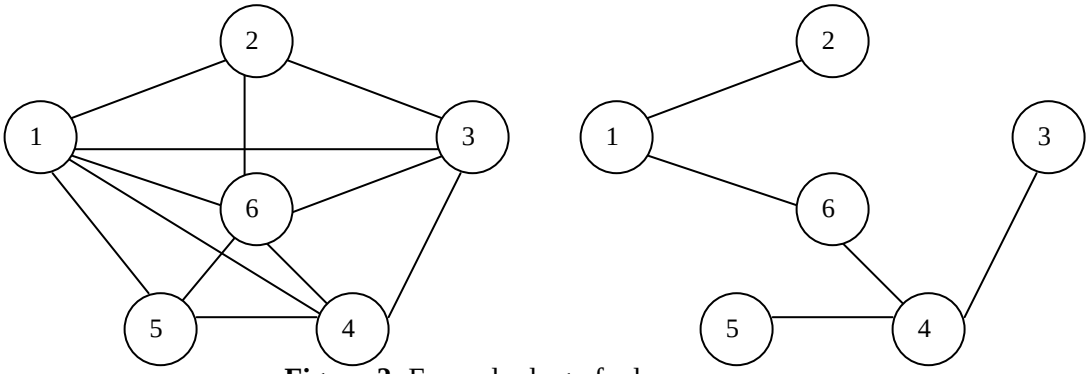


Figura 2: Exemplo de grafo denso e esparso

Saber se um grafo é denso ou esparso é importante na hora de definir as estruturas de dados para manipular o grafo, visto que a implementação usada para um grafo denso não é eficiente para um grafo esparso.

1.4. Grafos Direcionados

Até o momento, os grafos apresentados são bidirecionais, ou seja, se for possível ir do nó 1 para o 2, então também é possível realizar o caminho contrário. Em jogos isso nem sempre é possível. Considerando-se que exista uma porta com fechadura em apenas um lado, não será possível abrir a porta do outro lado. Assim, o grafo deve ter meio de representar esta informação. Outra situação ocorre quanto se pode navegar nos dois sentidos de uma aresta, porém com pesos diferenciados. Como exemplo, pode-se considerar a locomoção em um terreno acidentado. Para descrer a ladeira pode ser mais barato que subir (visto que pode gastar mais combustível ou tempo). Para estas situações, pode-se utilizar um grafo orientado (*Directed Graph - Digraph*). Se o grafo for orientado e não possuir ciclos, ele é chamado *Directed acyclic Graph*, ou DAG.

Uma aresta em um grafo orientado é um par ordenado que especifica a direção da aresta, que parte da posição origem e chega à posição destino.

Quando se trabalha com grafos não orientados, é conveniente se pensar em grafos orientados com duas arestas conectando cada par de vértices conectados. Isso é vantajoso, pois se pode usar a mesma estrutura de dados para representar ambos os tipos de grafos.

1.5. Uso de Grafos em Jogos

1.5.1. Grafo de navegação

O uso mais comum de grafos em jogos são os grafos de navegação. Eles representam uma abstração das posições onde os personagens podem se locomover no ambiente e a conexão entre essas posições (pontos). Deste modo, representa todos os possíveis caminhos que o personagem pode realizar. Essa informação é de vital importância para auxiliar o agente no planejamento do melhor caminho entre dois pontos quaisquer.

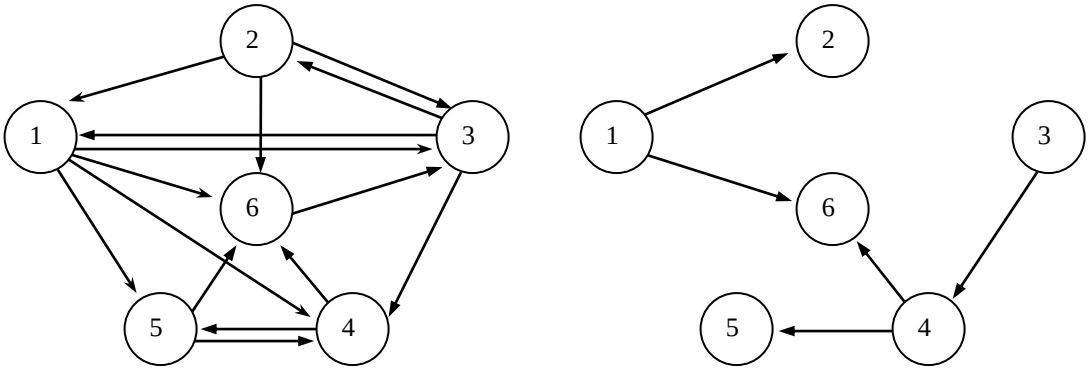


Figura 3: Exemplo de um grafo orientado

Cada nó em um grafo de navegação representa uma posição dentro do ambiente do jogo e cada aresta representa a conexão entre esses pontos. Adicionalmente, cada aresta pode ter um peso associado, que pode exemplo, representar a distância entre os pontos que a definem. Esse tipo de grafo é conhecido como grafo euclidiano. A Figura 4 apresenta um exemplo de um grafo de navegação de um ambiente 2D delimitado por paredes.

Deve-se observar que as arestas não são os únicos caminhos que o personagem pode percorrer. Eles representam caminhos que podem ser computados para encontrar o caminho de menor custo entre dois pontos quaisquer no cenário. Esse tipo de representação de grafo é muito utilizada em jogos do tipo FPS. Para jogos do tipo RTS, geralmente faz-se uso de uma grade de células, onde cada célula representa um tipo de terreno, como terra, água, grama, rocha, muros, etc. Assim, os grafos de navegação são criados usando o ponto central de cada célula. Os pesos são dados pelas distâncias e pelo tipo de terreno, por exemplo. Essa estratégia geralmente produz grafos que podem ser muito grandes, dependendo do tamanho do ambiente.

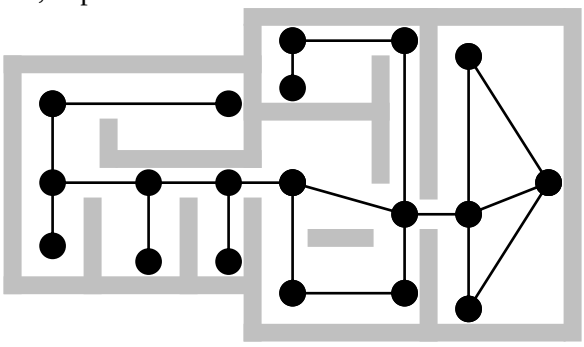


Figura 4: Exemplo de um Grafo de navegação

Em jogos, geralmente os vértices de um grafo são chamados de *waypoints*, ou pontos de caminho, visto que grafos em jogos são geralmente usados para representar caminhos que o personagem pode percorrer. Os *waypoints* são representados por vetores que indicam uma posição relativa ou absoluta no cenário do jogo. Podem ser utilizados para marcar um local a ser atingido, onde está localizado um determinado objeto ou para representar qualquer informação auxiliar para ajudar o personagem na locomoção em um ambiente qualquer.

1.5.2. Grafo de dependência

Grafos de dependência são usados em jogos que fazem uso de gerenciamento de recursos para descrever as dependências entre as várias construções, materiais, unidades e tecnologias disponíveis ao jogador. Na Figura 5 é apresentado um exemplo de grafo de dependência usado em jogos do tipo RTS.

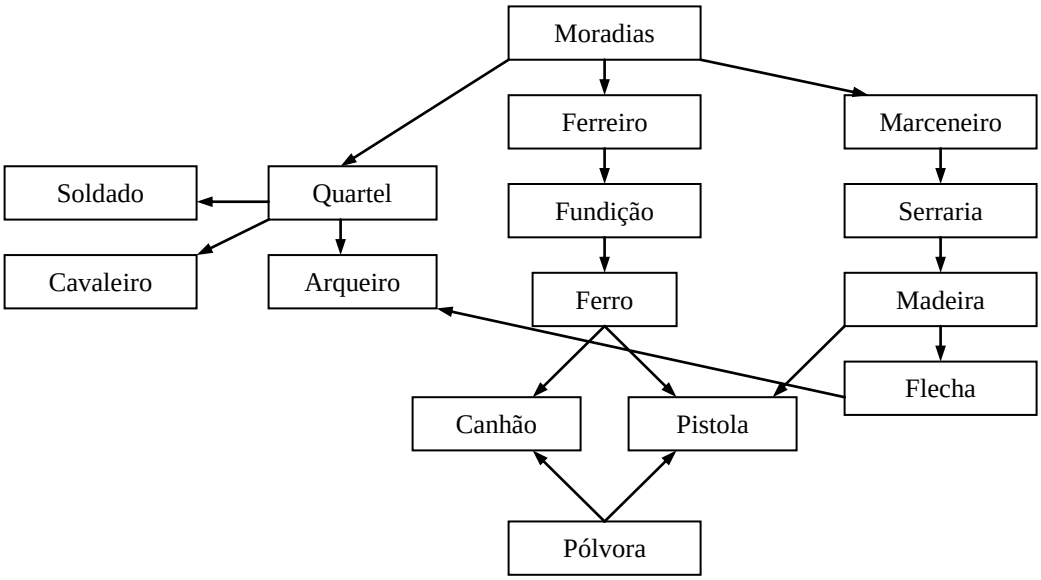


Figura 5: Exemplo de um grafo de dependência de um jogo RTS

Este grafo informa os pré-requisitos que são necessários para a criação de um dado recurso. A partir de um grafo de pendência, a IA do jogo pode decidir estratégias, prever o estado futuro do adversário, e utilizar recursos de forma eficiente. Eis alguns exemplos:

1. Se a IA está se preparando para uma batalha e verifica que arqueiros serão de grande valia, ela pode examinar o grafo de dependência e concluir que antes de gerar arqueiros, ela precisa ter certeza que já possui um quartel e tecnologia para fazer flechas. Ela também sabe que para produzir flechas ela precisa uma serraria para produzir madeira. Se a IA não tem nem o quartel nem a serraria, ela pode inspecionar o grafo de tecnologia para determinar que é mais importante construir o quartel antes da serraria. Isso porque o quartel é pré-requisito para a construção de 3 diferentes tipos de unidades de combate, enquanto a serraria é somente pré-requisito para produzir madeira. Se a batalha é iminente, a AI pode decidir investir recursos na construção de unidades de ataque tão logo quanto possível, porque todo mundo sabe que nesta hora soldados são mais importantes que madeira, por exemplo.
2. Se um soldado carregando uma pistola chega no território da IA, a IA pode processar no sentido contrário do grafo para concluir que:
    a. O inimigo já deve ter construído uma fundição e uma serraria
    b. O inimigo já deve ter desenvolvido a tecnologia da pólvora
    c. O inimigo deve estar produzindo recursos de ferro e madeira. Baseado nestas informações, a IA pode concluir que deve atacar a fundição ou a serraria, para acabar com a produção de armas do inimigo, ou de forma mais sorrateira, criar um assassino para tentar matar o ferreiro.
    d. Pode supor que o inimigo já tenha canhões ou os está construindo.
3. Se os custos de produção de cada item estão associados ao grafo de dependência, a IA pode se utilizar destes custos de produção para encontrar a rota mais eficiente para produzir cada recurso.

1.6. Implementação de Grafos

As estruturas mais comuns para representar grafos são matrizes de adjacências e Lista de adjacências. A matriz de adjacência são matrizes bidimensionais de booleanos ou valores reais que armazenam a informação de conectividade do grafo. Valores booleanos são usados quando não se tem um custo associado às arestas. Reais são usados quando se tem um custo para percorrer uma aresta, como no caso de distâncias entre nós.

Um exemplo de matriz de adjacência é ilustrado na Figura 6. Cada valor 1 representa a conexão entre dois nós e cada 0 representa a falta de conectividade. Como exemplo, **a partir do nó 3, pode-se ir ao nó 1 e ao nó 4, de forma direta**. Os indices da matriz representam o número do nó.

	0	1	2	3	4
0	0	0	0	1	0
1	0	0	0	0	0
2	1	0	0	1	1
3	0	1	0	0	1
4	1	1	0	0	0

Figura 6: Representação de um grafo por matriz de adjacência

Para representação dessa estrutura pode-se utilizar a seguinte estrutura:

```
typedef struct
{
   int m[N][N]; //essa matriz também poderia ser dinâmica, definida com int**;
   int tam; //dimensao da matriz m
}Grafo;
```

Ao invés de utilizar-se valores booleanos, pode-se associar pesos as arestas. Na seguinte tabela, pode-se ir do nó 3 para o nó 1 com peso 7.5. Valores 0, nessa representação, indicam falta de ligação entre os nós.

	0	1	2	3	4
0	0	0	0	2.01	0
1	0	0	0	0	0
2	10	0	0	1	1
3	0	-7.5	0	0	0.44
4	5.1	6.0	0	0	0

Figura 7: Representação de um grafo por matriz de adjacência

Apesar de ser uma representação intuitiva, apresenta problemas quanto à utilização da memória. Observa-se que em situações reais os grafos são geralmente esparsos e por isso uma melhor estrutura de representação é por meio de lista de adjacências. Nesta representação, para cada nó presente no grafo, existe uma lista com todos os demais nós que são atingíveis a partir dele. A figura 7 tem uma representação da Figura 6 por meio de listas de adjacências. Essa representação é mais eficiente na representação de grafos esparsos pois não perde espaço para armazenar conexões nulas, e por isso é a estrutura utilizada em jogos.

Algoritmos que operam sobre grafos devem ter um rápido acesso sobre os vértices e arestas. Para isso, o índice dos vértices no vetor de vértices deve ser o próprio índice do vértice, e o mesmo para a lista de adjacência de cada vértice.

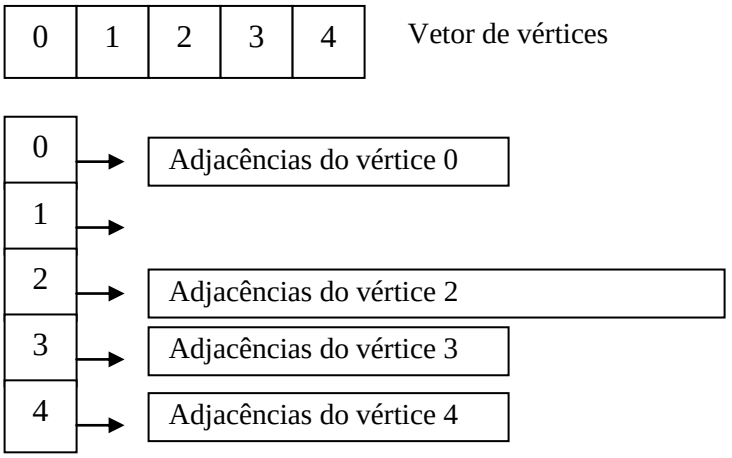


Figura 8: Vetor de Vértices e de lista de adjacências

Para representação dessa estrutura pode-se utilizar a seguinte estrutura:

```
typedef struct no
{
   struct no *prox; //vetor de ponteiro para uma lista encadeada de nós.
   int id;          //índice do no
}No;
typedef struct
{
   No *adjacencia[N]; //vetor de ponteiro para uma lista encadeada de nós.
                      //Cada nó contem um id
   int tam; //dimensao do vetor de ponteiros adjacencia
}Grafo;
```

Pode-se utilizar esta estrutura para se fazer a representação de árvores de qualquer tipo ou listas encadeadas, porém deve-se observar que não é a representação mais adequada. Em ambos os casos, deve-se indicar qual índice representa a raiz da árvore, ou em caso de listas encadeadas, qual é o primeiro nó.
Na seguinte tabela ilustra-se uma árvore binária composta por 5 nós. Adotou-se valores 1 para indicar ramos da esquerda e 2 para ramos a direita. O mesmo pode ser usado para listas duplamente encadeadas.

	0	1	2	3	4
0	0	0	0	0	0
1	0	0	0	0	0
2	1	0	0	0	0
3	0	1	0	0	2
4	1	0	2	0	0

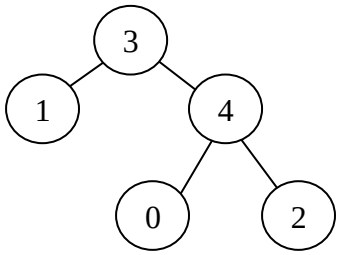


Figura 9: Representação de um grafo por matriz de adjacência

Como exemplo, dada uma árvore de qualquer tipo, é muito simples descobrir qual é o nó raiz. Basta achar qual o nó que não é alcançável por nenhum outro. Para isso deve-se encontrar o índice da coluna que tem todos os valores iguais a zero. O mesmo algoritmo pode ser usado para achar o nó raiz de uma lista encadeada. O seguinte programa ilustra esta tarefa.

```
int acha_raiz(char m[N][N], int N)
{
   for(int col=0; col<N; col++)
   {
      cont=0;
      for(int lin=0; lin<N; lin++)
      {
         if( m[lin][col] == 0 )
            cont++;
      }
      if (cont==N)
          return col;
   }
   return -1; //erro - não é uma árvore
}
```

Tendo-se o nó raiz da árvore, pode-se fazer o percorrimento nos nós. O seguinte exemplo ilustra uma implementação recursiva.

```
void print(char m[N][N], int raiz, int N)
{
   printf("%d", raiz);

   for(int col=0; col<N; col++)
   {
      if(m[raiz][col] != 0)
      {
         print(m, col, N);
      }
   }
}
```

Como exercícios, fazendo-se uso da estrutura de representação de grafos por meio de **matriz** e/ou **lista** de adjacências, implemente funções para fazer:

- o percorrimento dos nós de listas simplesmente encadeadas, listas duplamente encadeadas, listas circulares, árvores genéricas e árvores binárias de busca.
- para dizer se um grafo é uma árvore (binária ou genérica), ou uma lista (simples ou dupla).
- Encontrar a altura de uma árvore.

2. Tipos de Busca em Árvore/Grafo

1. Busca **Livre** em espaço de estados - Problemas / Quebra-cabeças
    1.1. Busca Cega (não informada)
        1.1.1. Busca em Profundidade (Depth-Search)
            - Algoritmo "Leve"
            - Pode não chegar na solução
            - Podemos inserir limite de profundidade
        1.1.2. Busca em Largura (Breadth-Search)
            - Algoritmo "pesado"
            - Deve chegar na solução (não se sabe quando)
            - A busca em largura é ótima se o custo dos passos for idêntico
            - Sempre encontra o caminho mais próximo
    1.2. Busca Heurística
        1.2.1. Branch-and-Bound Search
        1.2.2. A* Search
2. Busca **Condicionada** em espaços de estados
    • *Minimax*
    • *Minimax + Alpha-Beta*

2.1. Busca em Grafos

A escolha do algoritmo de busca em grafos depende inicialmente do tipo de informação presente no grafo. Se as arestas do grafo não possuírem peso, pode-se utilizar algoritmos de busca em largura ou profundidade. Se as arestas tiverem peso associado, pode-se usar os algoritmos de Dijkstra ou A*, ou Bellman-Ford caso pesos puderem assumir valores negativos.

Deve-se lembrar que o algoritmo de busca em profundidade sempre encontra um caminho, caso existir, porém não garante encontrar o caminho mais curto. Para isso, deve-se utilizar um algoritmo de busca em largura. O caminho mais curto é o que apresentar o menor número de nós.

Quando são considerados os pesos das arestas, o caminho mais curto não necessariamente é o caminho com menor número de nós, mas sim o que apresentar a menor soma dos pesos de um conjunto de arestas que parte da origem e chega ao destino.

O peso de um caminho p=<v_0, v_1, v_2 ... v_k> é a soma dos pesos de todas as arestas, ou seja,

$$w(p) = \sum_{i=1}^{k} w(v_{i-1}, v_i)$$

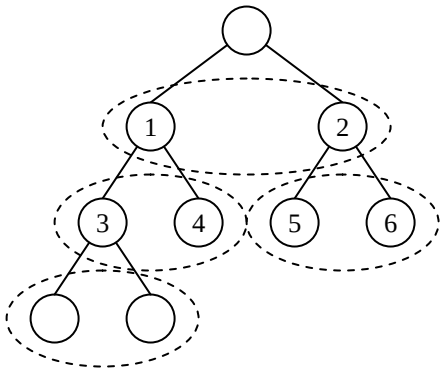
O menor caminho p entre u e v é dado por

$$\delta(u,v) = \begin{cases} \min\{(w(p) : u \xrightarrow{p} v\} \\ \infty \end{cases}$$

O menor caminho do vértice u ao vértice v é então definido como qualquer caminho p com peso $w(p) = \delta(u, v)$.

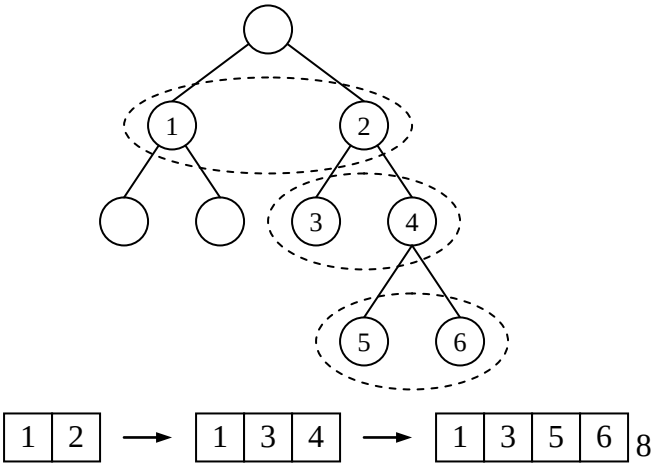
2.2. Busca em Largura não recursiva

```
function Busca_Largura (Inicio, Alvo)
{
   inserirFila(Inicio)
   while nao filaVazia()
   {
      no = removeFila()
      foiVisitado(no)
      if no == Alvo
      {
         return no
      }
      for each Filho in Expande(no)
      {
         if nao Visitado(Filho)
         {
            foiVisitado(Filho)
            insereFila(Filho)
         }
      }
   }
}
```



2.3. Busca em Profundidade

```
function Busca_Profundidade(Inicio, Alvo)
{
   empilha(Inicio)
   while nao pilhaVazia()
   {
      no = desempilha()
      foiVisitado(no)
      if no == Alvo
      {
         return no
      }
```



```
        for each Filho in Expande(no)
        {
            if nao Visitado(Filho)
            {
               foiVisitado(Filho)
               empilha(Filho)
            }
        }
    }
}
```

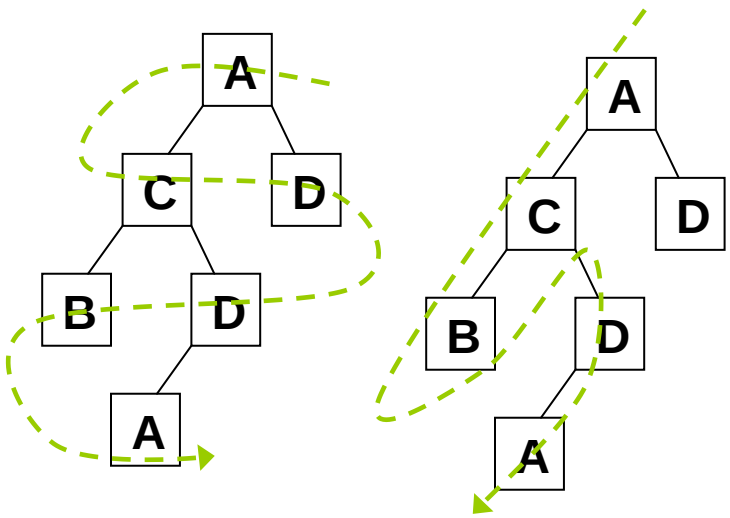


Figura 10: Exemplos de busca em largura e em profundidade

2.4. Problemas para discutir

How Chess Computers Work. By Marshall Brain for HowStuffWorks. "If you were to fully develop the entire tree for all possible chess moves, the total number of board positions is about 1, 000, 000, 000, 000, 000,000, 000, 000, 000,

algoritmo que serve para resolver o mesmo problema num grafo com pesos negativos é o algoritmo de Bellman-Ford. **É um algoritmo Guloso.**

Um exemplo prático de problema que pode ser resolvido pelo algoritmo de Dijkstra é: alguém precisa se deslocar de uma cidade para outra. Para isso, ela dispõe de várias estradas, que passam por diversas cidades. Qual delas oferece uma trajetória de menor caminho?

Este algoritmo produz gera uma árvore de caminho mais curto (*Shortest Path Tree* – SPT - *The Shortest Path Tree problem is to find the set of edges connecting all nodes such that the sum of the edge lengths from the root to each node is minimized*). Essa árvore é uma sub-árvore do grafo que representa o **caminho mais curto de um nó raiz a qualquer nó na STP**, como mostrado na Figura 10.

O algoritmo de Djikstra constrói a SPT uma aresta por vez, primeiramente adicionando o nó origem após a aresta que gera o caminho mais curto do nó origem ao nó ainda não presentes da SPT. Este algoritmo procede até que todos os nós sejam visitados. Se o algoritmo receber como entrada o nó destino, o processo termina tão logo este nó seja alcançado. Neste momento, a SPT contém o caminho mais curto do nó destino ao nó origem, e todos os demais nós visitados durante o processo.

Para o exemplo da Figura 10, consideremos que o nó inicial é o nó 5. Neste caso, o nó 5 é adicionado a SPT e as arestas que deixam 5 são colocadas na fronteira de busca. O algoritmo examina os nós destino das arestas que estão na fronteira (neste caso 6 e 2). O Nó 2 é o mais próximo e por isso é adicionado na SPT. Na seqüência, as arestas que deixam o nó 2 são adicionadas na fronteira de busca.

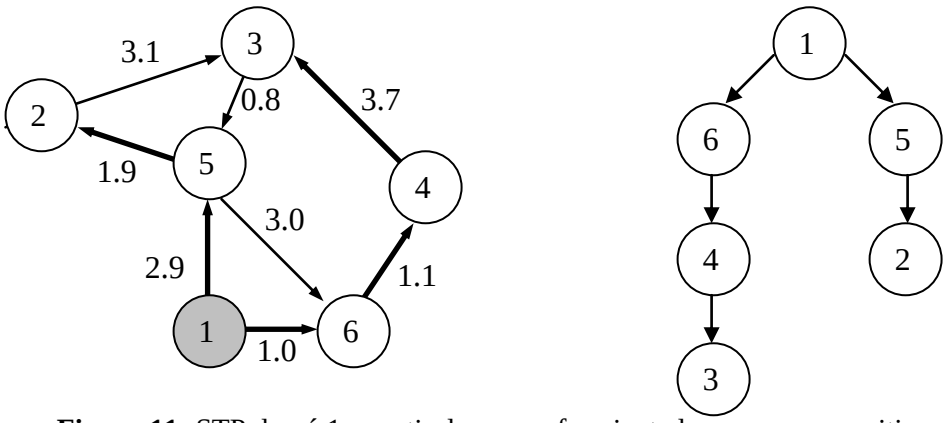


Figura 11: STP do nó 1 a partir de um grafo orientado com pesos positivos

Novamente o algoritmo examina o destino mais próximo a partir das arestas da fronteira. O custo para chegar ao nó 3 a partir do nó 5 é 5.0 e para o nó 6 é 3. Assim, o nó 6 é adicionado a SPT, e suas arestas na fronteira. Neste momento, têm-se duas arestas na fronteira: (2,3) e (6,4). O próximo nó a ser adicionado a SPT é o nó 4.

Têm-se então duas arestas que levam ao nó 3: indo por 4 ou por 2, sendo que o caminho (2,3) já está na fronteira. Examinando os dois caminhos, o algoritmo calcula que o caminho (5,2,3) tem custo 5 e que o caminho (5,6,4,3) tem custo 7.8. Assim a aresta (2,3) permanece na fronteira e a aresta (4,3) é removida. Assim, o vértice 3 é adicionado a SPT, visto que é o único nó na fronteira. A aresta (3,5) não é adicionada na fronteira pois o vértice 5 já esta na SPT. O nó 1 nunca foi considerado pois não existe nenhuma aresta que chegue a ele partindo no nó 5.

2.6. Algoritmo A*

O Algoritmo A* (A estrela) é muito semelhante ao algoritmo de Dijkstra. Porém, ao contrário do algoritmo de Dijkstra, este usa como estimativa da escolha do caminho não só o custo para chegar a um nó, mas também uma estimativa da distância ao destino. Esta estimativa é geralmente chamada de heurística.

A única diferença é o cálculo do custo dos custos dos nós que estão na fronteira de busca, que agora é dada por

$$f' = g + h'$$

onde g é o custo acumulado para atingir o nó e h' é uma estimativa heurística da distância real h até o nó destino. Assim, para o um nó N, que estava na fronteira, antecedido por P, e foi incluído na SPT, o custo associado ao nó é dado por

$$\texttt{Custo} = \texttt{CustoAcumulado(Origem, P)} + \texttt{Custo(P,N)} + \overbrace{\texttt{Custo(N, Destino)}}^{h'}$$

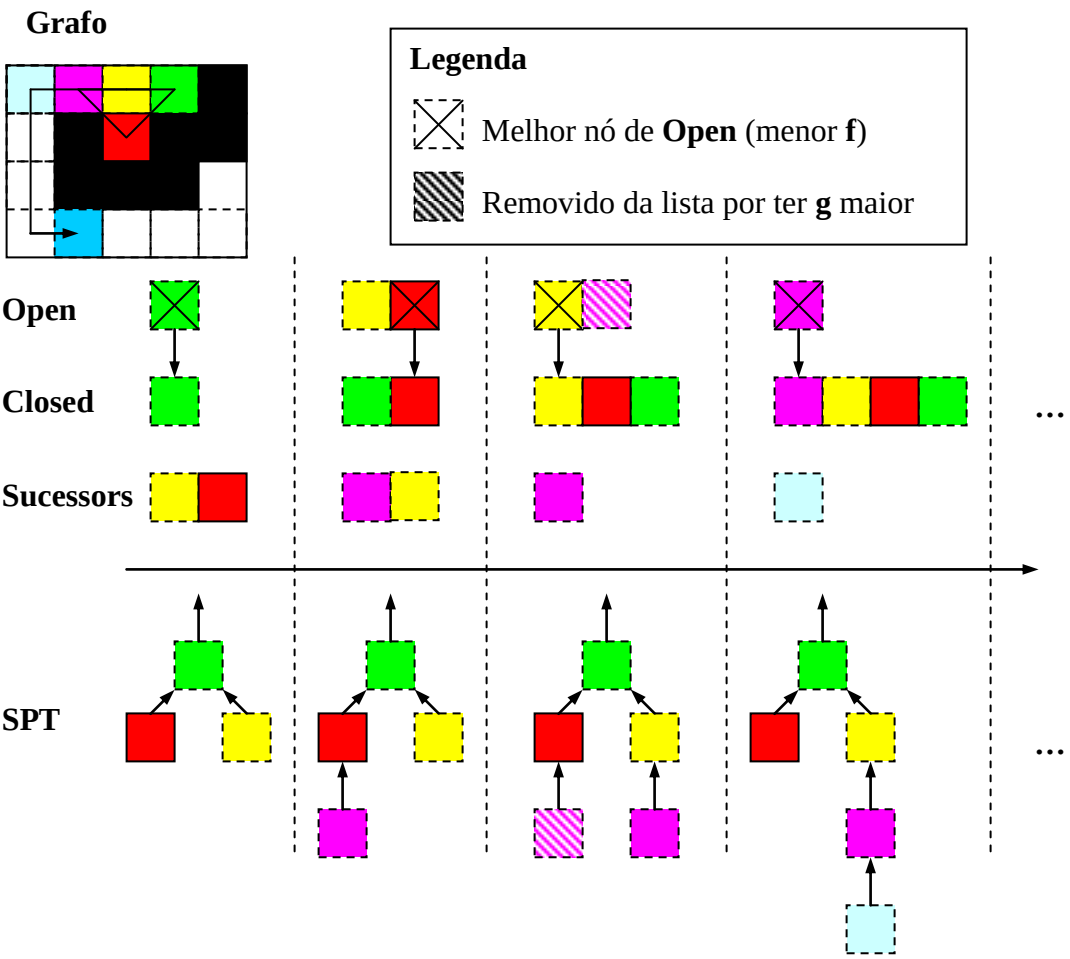
Isso faz com que a busca se concentre na direção do alvo, ao contrário do algoritmo de Dijkstra, que faz uma busca radial em todas as direções. Assim, menos nós podem ser avaliados, acelerando o processo de busca.

O cálculo da distância pode considerar a distância euclidiana (distância em linha reta) ao destino ou a distância *Manhattan*. Esta última pode ser aplicada a grafos de navegação com topologia em formato de grades, como no caso de jogos baseados em *tiles* (jogos de estratégia). A distância Manhattan é a soma dos deslocamentos em pedras (*tiles*) na horizontal e na vertical, ou seja, assume que é impossível percorrer um caminho andando na diagonal.

Algoritmo A*:
1. Sendo P o ponto de partida
2. Atribuía *f*, *g* e *h* a P
3. Adicione P na lista ABERTOS. Neste ponto, P é o único elemento da lista
4. Sendo M o melhor nó de ABERTOS (menor *f*)
    a) Se M é o destino, finalize e retorne o caminho
    b) Se ABERTOS é vazio, finalize e retorne erro
5. Para cada nó C conectado a M (**Lista de sucessores**)
    a) Atribua *f*, *g* e *h* à C
    b) Se C não está em ABERTOS ou FECHADOS
        i. Adicione C a Abertos
    c) Se C está em ABERTOS ou FECHADOS e tem **menor custo g**
        i. Retire C de ABERTOS ou FECHADOS
        ii. Adicione C a Abertos
6. Mova M de ABERTOS para FECHADOS e volte a etapa 4.

É necessário?

OBS:
→ Não deve ser confundido nó do grafo com nós usados nas listas encadeadas do algoritmo A*.
→ Para criar a **lista de sucessores**, deve-se alocar um conjunto de nós, onde cada nó deve ter um ponteiro para o pai, bem como deve-se definir os valores de f. g e h para este nó. O valor de g é dado pelo g do nó pai + o custo do nó pai até este nó.
→ Quando o algoritmo termina, a lista FECHADOS contém todos os nós acessados pelo algoritmo.
→ Para gerar o caminho, caso existir, deve-se pegar o nó M que seja o nó destino e percorrer pelo ponteiro pai até chegar ao nó de partida P.



Exercício: Elabore um exemplo onde é removido um nó da lista *Closed*. Teste o exemplo no algoritmo A*.

Algumas observações podem ser feitas sobre este algoritmo [4]:

A função g permite escolher qual nó será expandido, com base não apenas na qualidade do nó em si (medida por h'), mas também com base na qualidade do caminho até o nó. Incorporando g em f', nem sempre se escolhe como próximo nó a ser expandido o nó que parece estar mais próximo ao objetivo. Caso se queira escolher um caminho, pode-se definir **g sempre como 0**, escolhendo assim sempre o nó que parece estar mais próximo do objetivo. Para encontrar um caminho com **menor número de passos**, deve-se definir o custo de sair de um nó e chegar ao seu sucessor como uma constante, geralmente 1. Para encontrar o caminho mais barato, deve-se avaliar os pesos para sair de um nó e chegar a outro.

Em relação à estimativa da distância até o destino, dado por h, diferentes valores de h' podem gerar diferentes resultados. Se h' for um estimador perfeito, então o A* convergirá imediatamente para a solução ótima sem nenhuma busca. Quanto melhor for h', chega-se mais próximo da abordagem direta. Se **h' for sempre 0**, a estratégia de busca será controlada por g, e o algoritmo se comporta exatamente como o algoritmo de Dijkstra. Se o valor de **g for sempre 1**, a busca será em amplitude. Todos os nós de um nível terão valores mais baixos para g e, portanto, valores mais baixos para f', do que o nível seguinte. A principal regra é que h' nunca superestime o valor de h, ou seja, o custo estimado deve ser menor que o custo real. Assim, o A* certamente encontrará o caminho (determinado por g) para o objetivo, se existir.

Implementação:
O algoritmo utiliza 3 listas: ABERTOS, FECHADOS e SUCESSORES.

A lista ABERTOS contém os nós que já foram gerados e sofreram a aplicação da função heurística, mas que ainda não foram examinados (sem sucessores). É uma fila de prioridade onde os elementos de prioridade máxima são aqueles onde a função heurística tem valor mais promissor.

A lista FECHADOS contém nós já examinados. Esta lista é necessária pois em um grafo pode-se chegar a um nó por mais de um caminho, ao contrário de uma árvore.

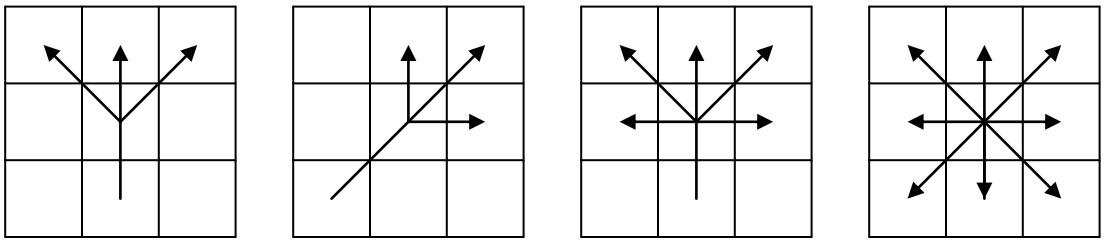
A lista SUCESSOR contém todos os nós que podem ser atingidos a partir do melhor nó da fila de ABERTOS.

Por ser um algoritmo pesado, técnicas de otimização se fazem necessárias. Para isso, pode-se fazer uso de uma implementação baseada principalmente no algoritmo *bucket sort*, visto a grande disponibilidade de memória RAM das máquinas atuais.

Define-se uma matriz de ponteiros com as dimensões do grafo para cada uma das listas ABERTOS e FECHADOS. Cada posição desta matriz ou é nulo ou aponta para o nó correspondente na lista encadeada, que contém as informações reais de cada nó visitado. Como o tempo de acesso a uma matriz é O(1), tem-se um acesso imediato a cada elemento das duas listas, caso existirem.

Com esta estratégia, operações de inserção, remoção e consulta de dados nas listas são realizadas em tempo O(1). A operação mais pesada é a de busca de menor peso (*f*) da lista, que gasta O(n). O algoritmo também faz uso de recursos para gerenciamento de alocação e desalocação de nós. Assim, quando um nó sai de uma lista e entra em outra, avalia-se se é necessário realocar outro nó ou não.

Para gerar a lista SUCESSOR, utiliza-se a função buildSucessors(), que tem como parâmetro o número de nós que podem ser visitados, considerando-se a direção de busca até então utilizada, como mostrado na seguinte figura. Esta solução pode ser usada para evitar a geração de caminhos com curvas muito bruscas.



(a) (b) (c) (d)

Como a implementação proposta visa ser didática, a lista de adjacências é gerada pela função buildSucessors(), que assume que cada nó pode ter até 8 vizinhos (8 arestas). Foi adotada esta estratégia pois se usa a matriz de vértices como um mapa de alturas, que além de guardar a vizinhança, também guarda o custo de cada aresta, dada pela diferença de altura entre nós vizinhos. Esta solução não permite dizer que o nó (1,1) tem ligação com o no (3,3), por exemplo. Isso somente seria possível se a matriz fosse 100x100, onde cada nó poderia ter uma aresta com cada um dos outros. Não são armazenadas explicitamente as listas de adjacências, como está representado nas classes GraphNode, GraphEdge e SparseGraph.

Nas Figuras 11 (grafo sem obstáculos) e 12 (grafo com obstáculos) são apresentados comparativos dos quatro algoritmos de busca tratados nesta seção: profundidade, largura, Dijkstra e A*. Observa-se que em todos os casos, o algoritmo A* achou o caminho com o menor número de nós visitados. Observa-se também que o algoritmo de busca em profundidade não encontra o melhor caminho e que o algoritmo de Dijkstra apresenta uma pequena redução do número de nós visitados em relação ao algoritmo de busca em largura. Para uma análise interativa em tempo real destes algoritmos, consulte [3].

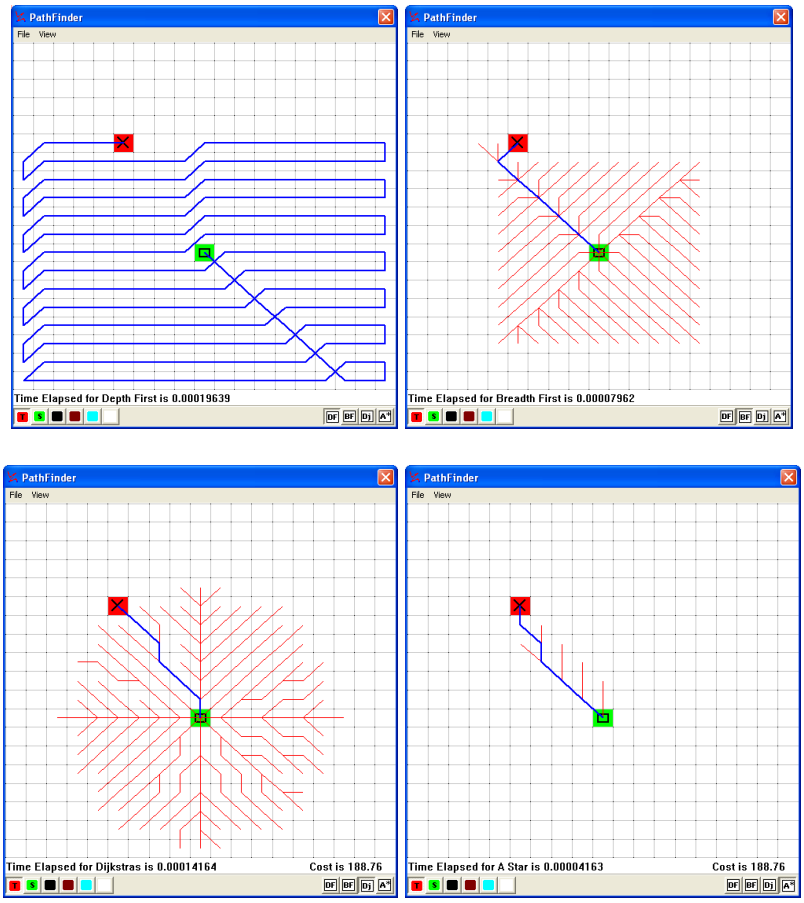


Figura 12: Exemplo de funções de busca para um grafo sem obstáculos [3]

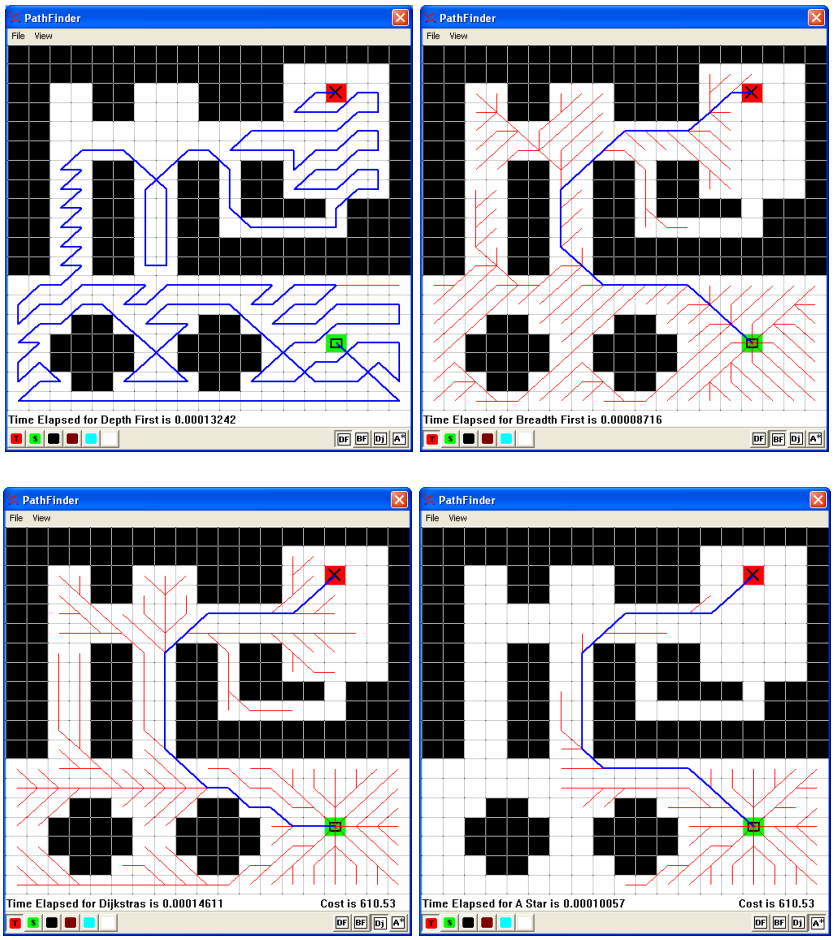


Figura 13: Exemplo de funções de busca para um grafo com obstáculos [3]

Exercícios:

1. Implementar os 4 algoritmos de busca em grafos e fazer análise de desepenho
2. Fazer animação que mostra a evolução a cada passo dos nós explorados por cada algoritmo.

Referências

[1] Antonio C. Mariani**. Conceitos Básicos da Teoria de Grafos**. Disponível em: http://www.inf.ufsc.br/grafos/definicoes/definicao.html

[2] Matt Buckland. **Programming Game AI by Example**. Wordware publishing Inc, 2005 (*Referência usada na maior parte deste material*)

[3] Matt Buckland. **Programming Game AI by Example**. Resource Page. Disponível em: http://www.wordware.com/files/ai/

[4] Rich, E., Knight, K. **Inteligência Artificial**. Makron Books, 1994.